



Lab #8 – More Branching

Conditional Jumps in Assembly:

Instruction	Description	Example
JE	Jumps if Equal (Zero Flag is set).	JE label2
JZ	Jumps if Zero Flag is set (same as JE).	JZ label3
JNE	Jumps if Not Equal (Zero Flag is clear).	JNE label4
JNZ	Jumps if Not Zero (same as JNE).	JNZ label5
JA	Jumps if Above (first operand > second, unsigned).	JA label6
JAЕ	Jumps if Above or Equal (unsigned comparison).	JAЕ label7
JB	Jumps if Below (first operand < second, unsigned).	JB label8
JBE	Jumps if Below or Equal (unsigned comparison).	JBE label9
JG	Jumps if Greater (first operand > second, signed).	JG label10
JGE	Jumps if Greater or Equal (signed comparison).	JGE label11
JL	Jumps if Less (first operand < second, signed).	JL label12
JLE	Jumps if Less or Equal (signed comparison).	JLE label13
JC	Jumps if Carry Flag is set (unsigned overflow).	JC label14
JNC	Jumps if Carry Flag is not set.	JNC label15
JO	Jumps if Overflow Flag is set (signed overflow).	JO label16
JNO	Jumps if Overflow Flag is not set.	JNO label17
JS	Jumps if Sign Flag is set (negative result).	JS label18
JNS	Jumps if Sign Flag is not set (non-negative result).	JNS label19

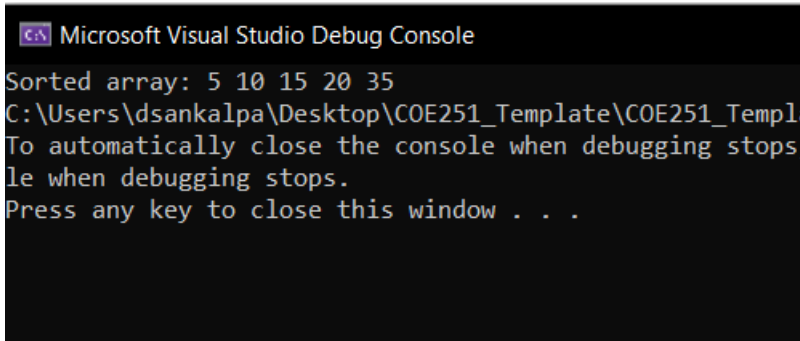
Shift and Rotate Instructions:

Instruction	Description	Example	Possible Operands
SHL	Shift Left: Shifts the bits of the destination operand to the left by a specified number of positions. Zeros are shifted into the least significant bits.	SHL EAX, 1	SHL reg, imm, SHL mem, imm, SHL reg, CL, SHL mem, CL
SHR	Shift Right: Shifts the bits of the destination operand to the right by a specified number of positions. Zeros are shifted into the most significant bits.	SHR EAX, 1	SHR reg, imm, SHR mem, imm, SHR reg, CL, SHR mem, CL

Shift instructions are useful in various bit manipulation operations such as multiplying/dividing by powers of 2 (SHL, SHR).

Lab Work (Questions 1-5):

1. Write an assembly program that sorts an array of 5 **unsigned** integers (myArray DWORD 20, 5, 35, 15, 10) in ascending order using a simple **bubble sort** algorithm. Use loops to iterate through the array and compare adjacent elements. If an element is larger than the next one, swap them. Repeat the process until the array is sorted.



```
Microsoft Visual Studio Debug Console
Sorted array: 5 10 15 20 35
C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Template
To automatically close the console when debugging stops,
press any key when debugging stops.
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Sorted Array: ", 0
str2 byte " ", 0
myArray dword 20,5,35,15,10
```

.code

main PROC

```
mov edx, offset str1
call writestring
```

```
mov ecx, lengthof myArray
```

```
sort:
dec ecx
mov ebx, 0
```

```
swap:
mov eax, myArray[ebx * 4]
mov edx, myArray[ebx * 4 + 4]
```

```
cmp eax, edx
jbe no_swap
```

```
mov myArray[ebx * 4], edx
mov myArray[ebx * 4 + 4], eax
```

```
no_swap:
inc ebx
cmp ebx, ecx
```

```
j1 swap
```

```
cmp ecx, 1
jg sort
```

```

    mov ecx, lengthof myArray
    mov ebx, 0

print:
    mov eax, myArray[ebx * 4]
    call writeDec

    mov edx, offset str2
    call writestring

    inc ebx
    cmp ebx, ecx
    jl print

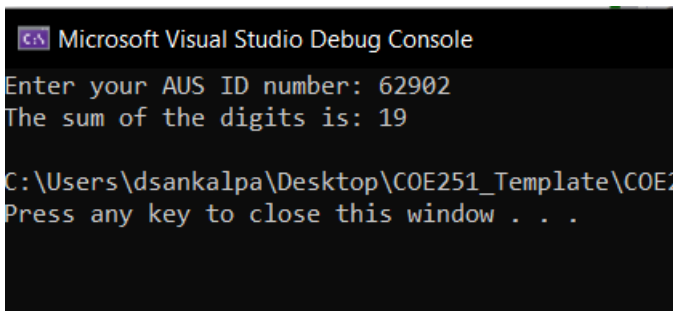
    exit
main ENDP
    END main

```

Screenshot:

Sorted Array: 5 10 15 20 35

2. Write an assembly program that reads your AUS ID number (an unsigned number) and calculates the sum of its digits and prints them.



Solution:

```

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    str1 byte "Enter your AUS ID number: ", 0
    str2 byte "The sum of the digits is: ", 0

.code
main PROC
    mov edx, offset str1
    call writestring
    call readint

    mov ebx, 0
    mov ecx, eax

    sum_digit:
    test ecx, ecx
    jz print_sum

    mov edx, 0
    mov eax, ecx

```

```

mov esi, 10
div esi

add ebx, edx

mov ecx, eax
jmp sum_digit

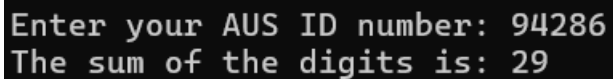
print_sum:
mov edx, offset str2
call writestring
mov eax, ebx
call writedec

call crlf

exit
main ENDP
END main

```

Screenshot:



```

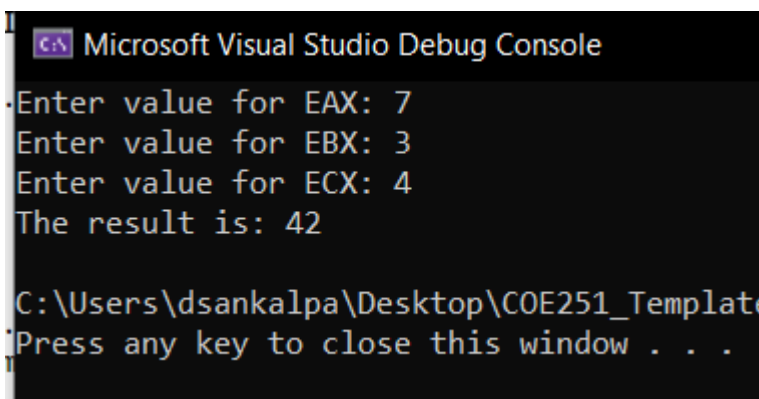
Enter your AUS ID number: 94286
The sum of the digits is: 29

```

- Write an assembly program to calculate the following mathematical expression using only shifts, additions, and subtractions (do not use MUL or DIV or LOOP instructions):

$$Result = (5 \times EAX + 3 \times EBX) - \frac{(2 \times ECX)}{4}$$

Ask the user to enter the three values.



```

Microsoft Visual Studio Debug Console
Enter value for EAX: 7
Enter value for EBX: 3
Enter value for ECX: 4
The result is: 42

C:\Users\dsankalpa\Desktop\COE251_Template
Press any key to close this window . . .

```

Solution:

TITLE Name/Purpose Here

```
INCLUDE Irvine32.inc
```

```
.data
```

```

str1 byte "Enter value for EAX: ", 0
str2 byte "Enter value for EBX: ", 0
str3 byte "Enter value for ECX: ", 0
str4 byte "The result is: ", 0
a_val dword ?

```

```

    b_val dword ?
    c_val dword ?
    rst dword ?

.code
main PROC
    mov edx, offset str1
    call writestring
    call readint
    mov a_val, eax

    mov edx, offset str2
    call writestring
    call readint
    mov b_val, eax

    mov edx, offset str3
    call writestring
    call readint
    mov c_val, eax

    mov eax, a_val
    shl eax, 2
    add eax, a_val

    mov ebx, b_val
    shl ebx, 1
    add ebx, b_val

    add eax, ebx

    mov ecx, c_val
    shl ecx, 1

    shr ecx, 2

    sub eax, ecx

    mov rst, eax

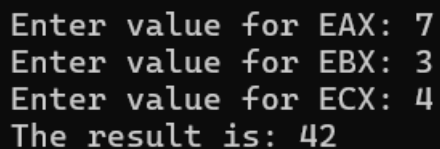
    mov edx, offset str4
    call writestring
    mov eax, rst
    call writedec

    call crlf

    exit
main ENDP
    END main

```

Screenshot:

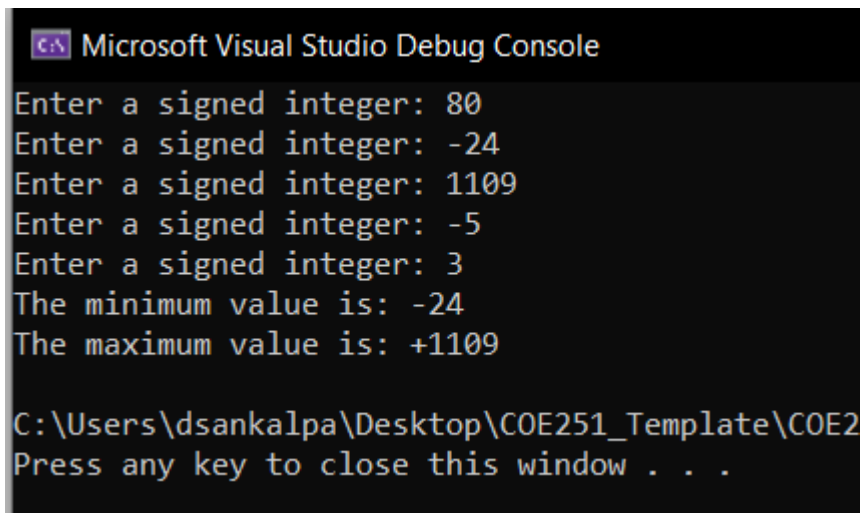


```

Enter value for EAX: 7
Enter value for EBX: 3
Enter value for ECX: 4
The result is: 42

```

4. Write an assembly program that reads 5 signed integers from the user and finds both the minimum and maximum values.

A screenshot of the Microsoft Visual Studio Debug Console window. The title bar reads "Microsoft Visual Studio Debug Console". The console output shows the following text: "Enter a signed integer: 80", "Enter a signed integer: -24", "Enter a signed integer: 1109", "Enter a signed integer: -5", "Enter a signed integer: 3", "The minimum value is: -24", "The maximum value is: +1109", and "C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Template.asm". At the bottom, it says "Press any key to close this window . . .".

```
Microsoft Visual Studio Debug Console

Enter a signed integer: 80
Enter a signed integer: -24
Enter a signed integer: 1109
Enter a signed integer: -5
Enter a signed integer: 3
The minimum value is: -24
The maximum value is: +1109

C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Template.asm
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter a signed integer: ", 0
str2 byte "The minimum value is: ", 0
str3 byte "The maximum value is: ", 0
arr dword 5 DUP(?)
```

.code

main PROC

```
mov esi, 0
```

```
mov ecx, lengthof arr
```

```
again:
```

```
mov edx, offset str1
```

```
call writestring
```

```
call readint
```

```
mov [arr + esi], eax
```

```
add esi, 4
```

```
loop again
```

```
mov edx, offset str2
```

```
call writestring
```

```
mov esi, 4
```

```
mov ebx, [arr]
```

```
mov edi, [arr]
```

```
mov ecx, 4
```

```
first:
```

```
cmp ebx, [arr + esi]
```

```
jge max
```

```
mov ebx, [arr + esi]
```

```
max:
```

```
add esi, type arr
```

```
loop first
```

```

mov esi, 4
mov ecx, 4

second:
cmp edi, [arr + esi]
jle min

mov edi, [arr + esi]

min:
add esi, type arr
loop second

mov eax, edi
call writeint

call crlf

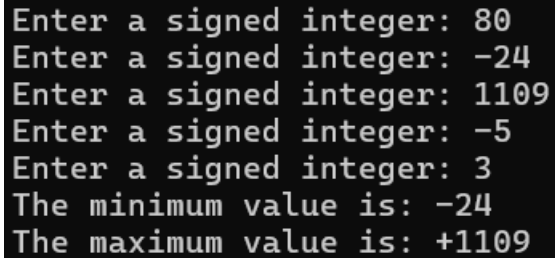
mov edx, offset str3
call writestring
mov eax, ebx
call writeint

call crlf

exit
main ENDP
END main

```

Screenshot:

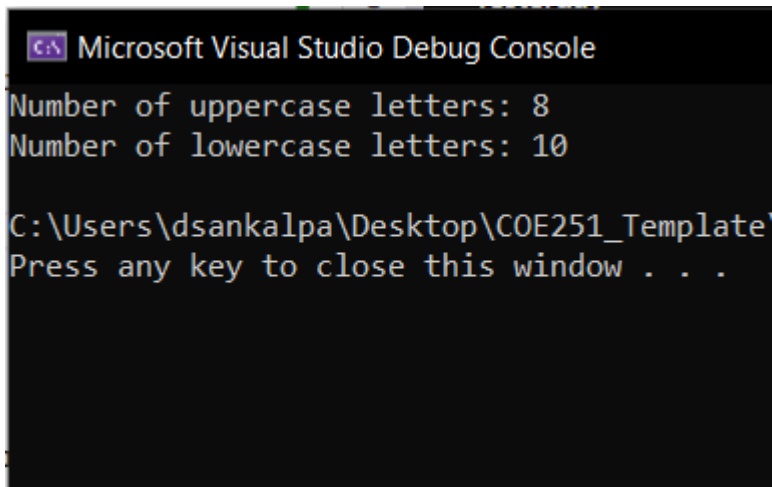


```

Enter a signed integer: 80
Enter a signed integer: -24
Enter a signed integer: 1109
Enter a signed integer: -5
Enter a signed integer: 3
The minimum value is: -24
The maximum value is: +1109

```

5. Write an assembly program that counts the number of uppercase and lowercase letters in myString BYTE "HeLlo aSsEmBlY wOrLd", 0



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Number of uppercase letters: ", 0
str2 byte "Number of lowercase letters: ", 0
myString byte "HeLlo aSsEmBlY wOrLd", 0
upperSum byte 0
lowerSum byte 0
```

.code

main PROC

```
mov edx, offset str1
call writestring
```

```
mov esi, 0
mov ecx, 20
```

```
again:
movzx eax, byte ptr [myString + esi]
```

```
cmp eax, 32
je end_program
cmp eax, 97
```

```
jae lowercase
```

```
inc upperSum
```

```
jmp end_program
```

```
lowercase:
inc lowerSum
```

```
end_program:
inc esi
loop again
```

```
movzx eax, upperSum
call writedec
```

```
call crlf
```

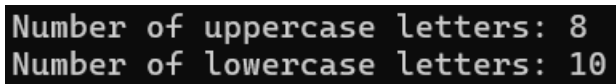


```
    mov edx, offset str2
    call writestring
    movzx eax, lowerSum
    call writedec

    call crlf

    exit
main ENDP
    END main
```

Screenshot:



Number of uppercase letters: 8
Number of lowercase letters: 10